25

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

ThreadPoolExecutor Anatomisi: CorePool, MaxPool, Kuyruk & Büyüme Algoritması

ThreadPoolExecutor Anatomy: Core vs Max Pool, Queues & Growth Dynamics

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              THREADPOOLEXECUTOR GÖREV KABUL KARAR AĞACI  │
└──────────────────────────────────────────────────────────┘

YENİ BİR GÖREV GELDİ (execute(task)):

1. ADIM: Çalışan thread sayısı < corePoolSize mı?
   ├──► EVET: Hemen yeni bir Core Thread aç ve görevi başlat!
   └──► HAYIR: 2. Adıma geç.

2. ADIM: BlockingQueue (İş Kuyruğu) dolu mu?
   ├──► HAYIR: Görevi kuyruğa at (Mevcut boştaki işçiler alsın).
   └──► EVET: 3. Adıma geç.

3. ADIM: Çalışan thread sayısı < maximumPoolSize mı?
   ├──► EVET: Hemen geçici bir Max Thread aç ve görevi başlat!
   └──► HAYIR: 4. Adıma geç.

4. ADIM: REJECTION POLICY (RET POLİTİKASI) DEVREYE GİRER!
   └──► (Abort, CallerRuns, Discard veya DiscardOldest)

⚠ EN BÜYÜK YANILGI: Havuz doğrudan maximumPoolSize'a kadar büyümez!
Önce kuyruk DOLMAK zorundadır! Kuyruk dolmadan ek thread ASLA açılmaz!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kargo şubesini düşünün. Şubede 3 daimi kadrolu personel (corePoolSize) çalışır. İlk 3 kargo gelince hemen bu personellere verilir. 4. kargo geldiğinde müdür yeni personel tutmaz; kargoyu arkadaki depoya (BlockingQueue) koyar! Ancak depo ağzına kadar dolup taştığında (Kuyruk dolduğunda), müdür geçici acil durum personelleri (maximumPoolSize) tutmaya başlar. Eğer depo da dolu, tüm geçici personeller de meşgulse; kapıya gelen müşteriye 'Kargonuzu alamıyoruz' denir (Rejection Policy)!

🇬🇧 ENGLISH (ELI5)

Think of a busy post office. It employs 3 full-time clerks (`corePoolSize`). The first 3 parcels go directly to them. When the 4th parcel arrives, the manager does NOT hire new staff; the parcel is placed into the backroom storage shelf (`BlockingQueue`). Only when that shelf is 100% physically packed full does the manager hire auxiliary temporary workers (`maximumPoolSize`). If temporary workers hit the legal ceiling AND the shelf is overflowing, incoming parcels are turned away at the door (`RejectedExecutionHandler`)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ThreadPoolExecutor Parametreleri ve HotSpot Yaşam Döngüsü: 1) corePoolSize vs maximumPoolSize: Birçok mühendis görevler geldikçe havuzun `maximumPoolSize`'a kadar büyüyeceğini sanır; bu yanlıştır! Standart mantıkta görev önce kuyruğa konur. Yalnızca kuyruk DOLDUĞUNDA `maximumPoolSize`'a kadar yeni thread açılır. 2) keepAliveTime ve Unit: Boşta kalan (idle) geçici thread'lerin ne kadar süre sonra sonlandırılacağını belirler (`allowCoreThreadTimeOut(true)` ile core thread'ler de sonlandırılabilir). 3) ctl Atomik Değişkeni: Doug Lea, havuz durumunu (RUNNING, SHUTDOWN, STOP, TIDYING, TERMINATED) ve aktif thread sayısını tek bir 32-bit `AtomicInteger ctl` içinde bit-masking ile tutar (Üst 3 bit durum, alt 29 bit thread sayısı).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ThreadPoolExecutor Architecture & Bit-Packing: 1) The Growth Order Axiom: Tasks route to: `corePool -> workQueue -> maxPool -> RejectedExecutionHandler`. Extra threads past `corePoolSize` are ONLY created if the queue rejects the `offer()`. 2) Atomic Control Word (`ctl`): HotSpot uses a single `AtomicInteger ctl` to pack both worker count and run state. The high 3 bits represent states (RUNNING = −1, SHUTDOWN = 0, STOP = 1, TIDYING = 2, TERMINATED = 3); the lower 29 bits encode the active worker count (supporting up to 536 million threads). 3) Worker Encapsulation: Internal `Worker` classes extend AQS to implement non-reentrant mutual exclusion locks, preventing interrupts during active task execution.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Üretim Kalitesinde ThreadPoolExecutor Konfigürasyonu
import java.util.concurrent.*;

public class ProductionThreadPoolFactory {
    public static ThreadPoolExecutor createCustomPool() {
        int corePoolSize = 4;
        int maxPoolSize = 8;
        long keepAliveTime = 60L;

        // ⚠ Sınırlı (Bounded) kuyruk OOM'u kesinlikle önler!
        BlockingQueue<Runnable> queue = new ArrayBlockingQueue<>(100);

        ThreadFactory threadFactory = Thread.ofPlatform()
            .name("order-processor-", 1)
            .factory();

        // 4. Adım: Kuyruk ve Max dolunca çağıran thread çalıştırsın (CallerRuns)
        RejectedExecutionHandler handler = new ThreadPoolExecutor.CallerRunsPolicy();

        return new ThreadPoolExecutor(
            corePoolSize,
            maxPoolSize,
            keepAliveTime,
            TimeUnit.SECONDS,
            queue,
            threadFactory,
            handler
        );
    }

    void main() {
        var pool = createCustomPool();
        pool.execute(() -> IO.println("Görev çalıştı: " + Thread.currentThread().getName()));
        pool.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı Sorusudur: 'corePoolSize=2, maxPoolSize=10, queueCapacity=100 olan bir havuza aynı anda 50 görev gönderirsek kaç thread açılır?' Yanıt: 'SADECE 2 THREAD AÇILIR! Çünkü ilk 2 görev core thread'leri alır, kalan 48 görev 100 kapasiteli kuyruğa atılır. Kuyruk dolmadığı için 3. thread bile açılmaz!' Bu soruyu bilen aday doğrudan öne geçer.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

The 1 MAANG Pool Question: 'If core=2, max=10, and queueCapacity=100, and 50 tasks arrive concurrently, how many threads exist?' Answer: 'EXACTLY 2 THREADS! The first 2 tasks spawn the core workers. The remaining 48 tasks are buffered in the 100-capacity queue. Because the queue never overflows, temporary workers are never spawned.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Havuz kuyruk dolmadan maximumPoolSize'a kadar ASLA genişlemez.
- ctl değişkeni tek bir 32-bit integer içinde hem havuz durumunu hem thread sayısını tutar.
- Üretimde sınırsız (unbounded) kuyruk kullanmak OOM çöküşlerine davetiyedir.

🎯 Key Takeaways for the Interview

- Pools never expand past corePoolSize until the bounded work queue is 100% full.
- HotSpot packs pool run state and thread count into a single 32-bit atomic integer.
- Unbounded work queues cause OutOfMemoryErrors under sustained traffic spikes.